De Hadoop/MapReduce à l'optimisation avec Spark (Docker & PySpark)

1. Objectifs du

À la fin de ce TP, vous serez capables de :

- Comprendre les limites historiques du stockage distribué et du moteur MapReduce ;
- Lancer un environnement complet de Big Data avec **Docker** ;
- Initialiser une SparkSession et manipuler vos premiers **DataFrames** ;
- Observer et maîtriser la gestion des partitions ;
- Manipuler l'évaluation paresseuse (*lazy evaluation*) et lire l'architecture d'exécution (*Jobs, Stages, Tasks*) via la **Spark UI**.

Etape 2 : mapreduce (que jai supprimé)

3. Étape par étape : Environnement et Premiers pas avec Spark

Étape 3.1 — Vérifier Docker

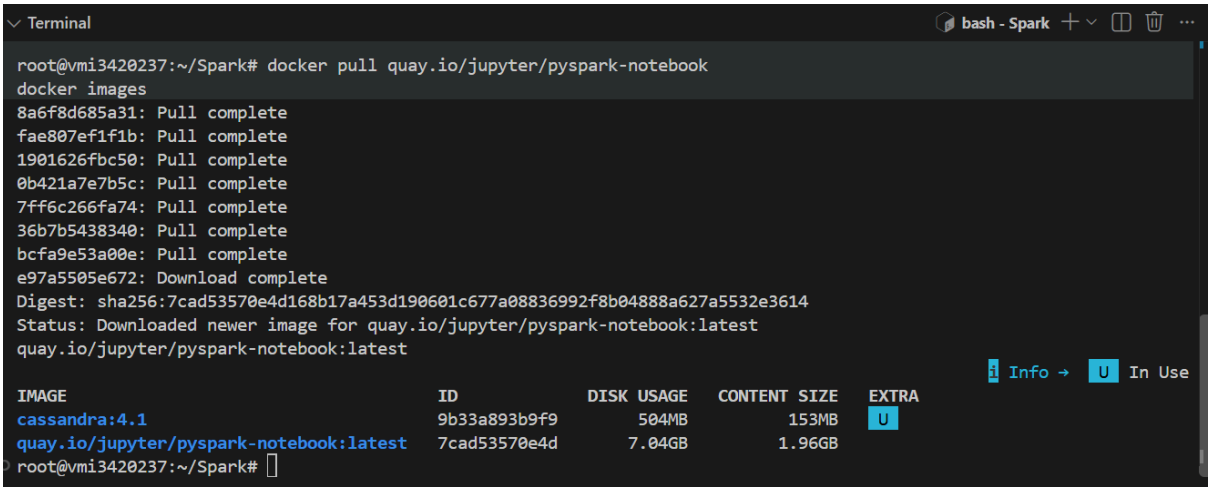
Assurez-vous que Docker est bien lancé sur votre machine, puis vérifiez son bon fonctionnement dans votre terminal :

```
docker --version
docker run hello-world
```

Étape 3.2 — Télécharger l'image PySpark

Nous utilisons l'image officielle des *Jupyter Docker Stacks*, qui embarque tout le nécessaire (Jupyter, Python, Spark) :

```
docker pull quay.io/jupyter/pyspark-notebook
```

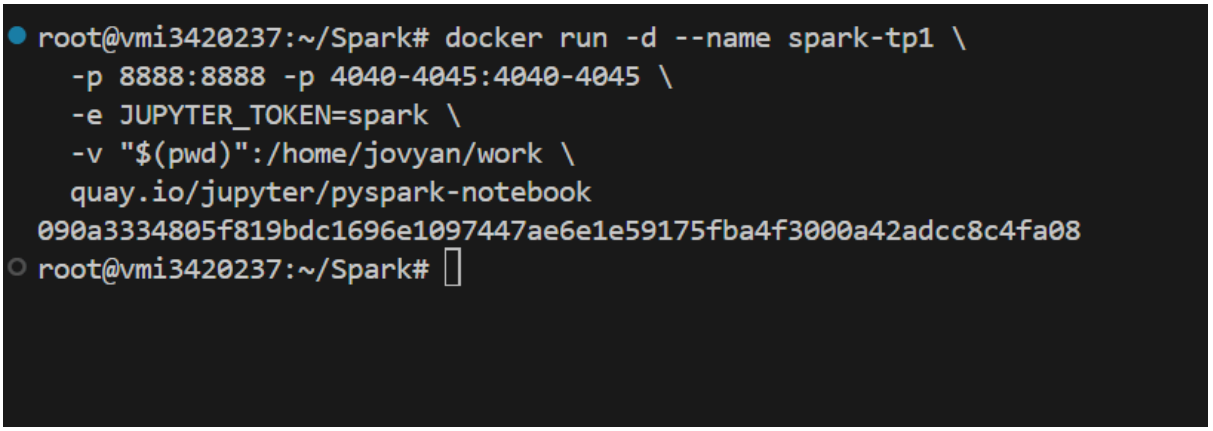


Étape 3.3 — Lancer le conteneur de travail

Exécutez la commande suivante pour démarrer le conteneur en arrière-plan (`-d`), en exposant Jupyter sur le port `8888` et la Spark UI sur la plage `4040-4045`

Sous Linux Bash :

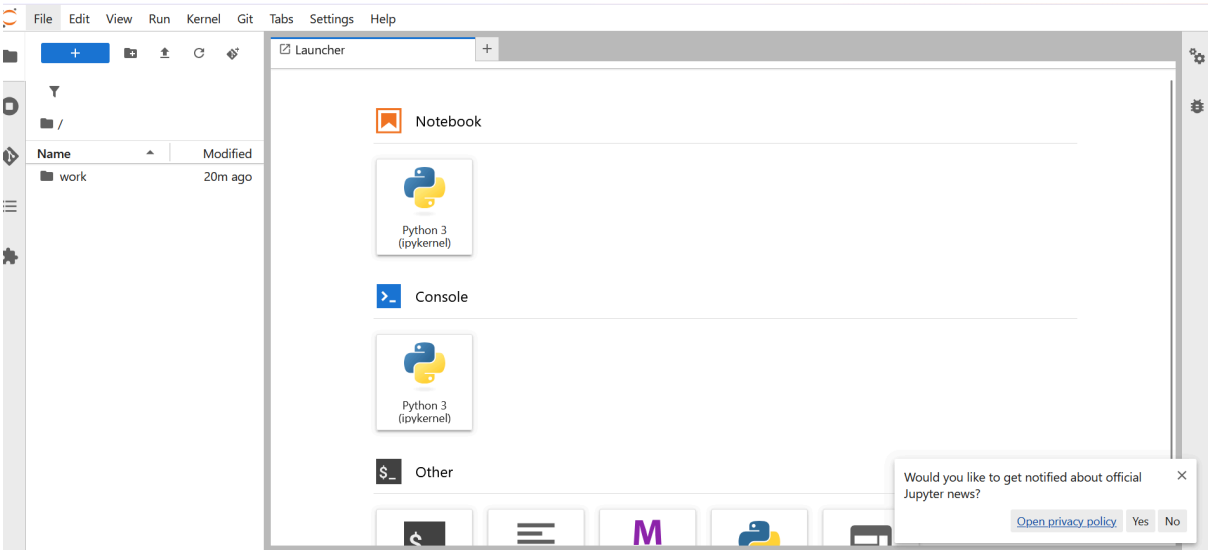
```
docker run -d --name spark-tp1 \
  -p 8888:8888 -p 4040-4045:4040-4045 \
  -e JUPYTER_TOKEN=spark \
  -v "$(pwd)":/home/jovyan/work \
  quay.io/jupyter/pyspark-notebook
```



Vérifiez que le conteneur est bien actif avec `docker ps` (état **Up**).

Étape 3.4 — Ouvrir Jupyter

1. Ouvrez votre navigateur et rendez-vous sur **http://localhost:8888/?token=spark**.
2. Entrez dans le dossier **work** et créez un nouveau notebook Python 3 nommé tp1_decouverte_spark.ipynb



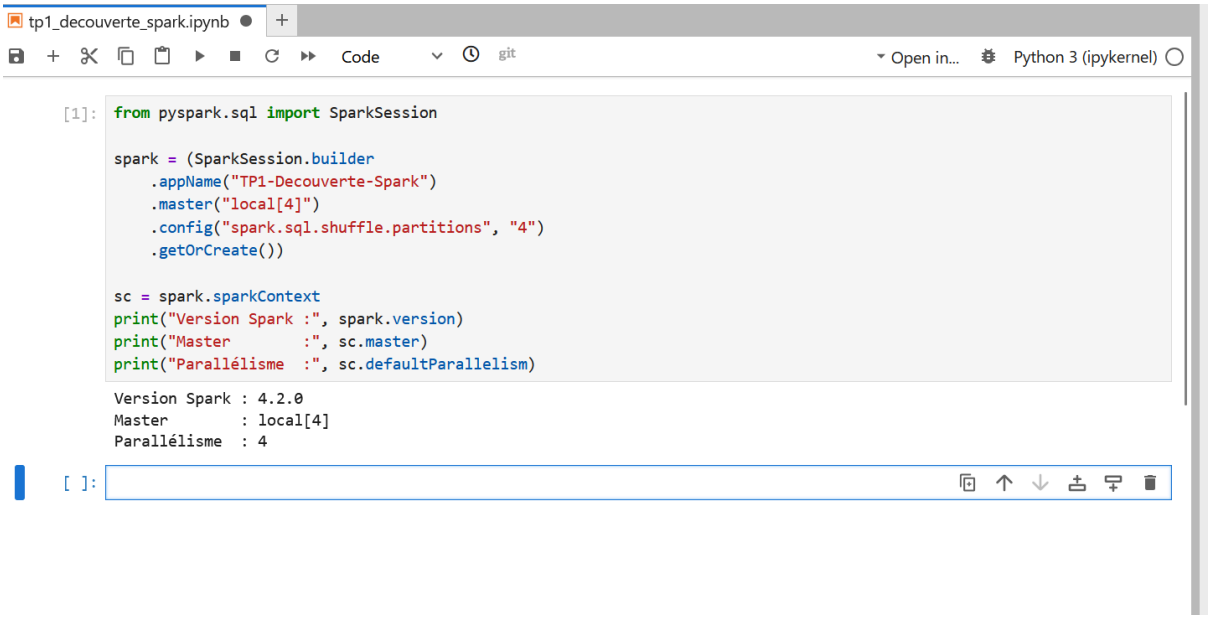
4. Initialisation de la SparkSession

Dans la première cellule de votre notebook, écrivez le code d'initialisation :

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
    .appName("TP1-Decouverte-Spark")
    .master("local[4]")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate())

sc = spark.sparkContext
print("Version Spark :", spark.version)
print("Master        :", sc.master)
print("Parallélisme  :", sc.defaultParallelism)
```



master("local[4]") : Précise que Spark s'exécute en local sur votre machine en utilisant 4 threads. Le Driver et les Executors s'exécutent dans le même processus.
spark.sql.shuffle.partitions = 4 : Réduit la valeur par défaut (initialisée à 200) pour garder des affichages lisibles sur notre petit jeu de données d'apprentissage.

5. Manipulation des premiers DataFrames

A. Génération d'un DataFrame natif

Testons la création d'un DataFrame de test généré automatiquement par Spark :

```
myRange = spark.range(1000).toDF("number")
myRange.show(5)
myRange.printSchema()
print("Nombre total de lignes :", myRange.count())
```



B. Création d'un DataFrame à partir de données structurées

Créons un ensemble de données tabulaires :

```
data = [("Alice", 29), ("Bob", 31), ("Catherine", 35)]
df = spark.createDataFrame(data, ["name", "age"])

df.show()
df.printSchema()
```

```
[4]: data = [("Alice", 29), ("Bob", 31), ("Catherine", 35)]
     df = spark.createDataFrame(data, ["name", "age"])

     df.show()
     df.printSchema()
```

```
+---------+---+
|     name|age|
+---------+---+
|    Alice| 29|
|      Bob| 31|
|Catherine| 35|
+---------+---+

root
 |-- name: string (nullable = true)
 |-- age: long (nullable = true)
```

Exercice d'application :

Ajoutez trois nouvelles personnes à vos données : **David (41), Emma (27), Frank (38)**. Puis réalisez les opérations suivantes dans votre notebook :

1. Affichez le DataFrame mis à jour ;
2. Affichez son schéma pour vérifier les types de données ;
3. Comptez le nombre total de lignes (le résultat attendu est 6) ;
4. Affichez uniquement la colonne des prénoms (name) ;
5. Filtrez et affichez uniquement les personnes dont l'âge est strictement supérieur à 30 ans.

```
[5]: from pyspark.sql.functions import col

data += [("David", 41), ("Emma", 27), ("Frank", 38)]
df = spark.createDataFrame(data, ["name", "age"])

df.show()
df.printSchema()
print("Total :", df.count())
df.select("name").show()
df.filter(col("age") > 30).show()
```

```
+---------+---+
|     name|age|
+---------+---+
|    Alice| 29|
|      Bob| 31|
|Catherine| 35|
|    David| 41|
|     Emma| 27|
|    Frank| 38|
+---------+---+

root
 |-- name: string (nullable = true)
 |-- age: long (nullable = true)

Total : 6
```

6. Architecture, Partitions et Parallélisme

A. Observer les partitions

Dans Spark, les données d'un DataFrame sont découpées en morceaux appelés **partitions**. Chaque partition est traitée de manière indépendante par une tâche (*task*).

Exécutez ce code pour observer les partitions de votre DataFrame :

print("Nombre de partitions :", df.rdd.getNumPartitions())

print("Contenu réparti :", df.rdd.glom().collect())

```
[6]: print("Nombre de partitions :", df.rdd.getNumPartitions())
     print("Contenu réparti :", df.rdd.glom().collect())
```

```
Nombre de partitions : 4
Contenu réparti : [[Row(name='Alice', age=29)], [Row(name='Bob', age=31), Row(name='Catherine', age=35)], [Row(name
='David', age=41)], [Row(name='Emma', age=27), Row(name='Frank', age=38)]]
```

La fonction `glom()` permet de regrouper les lignes par partition physique.

Partition 1 : [Alice , Bob , Catherine]

Partition 2 : [David , Emma , Frank]

B. Modifier le nombre de partitions

Il est possible de redistribuer les données en modifiant le nombre de partitions :

df2 = df.repartition(2)

```
print("Nouvelles partitions :", df2.rdd.getNumPartitions())
```

7. L'Évaluation Paresseuse (*Lazy Evaluation*) et le DAG

L'une des forces majeures de Spark est son mode d'exécution paresseux.

- Les **transformations** (comme filter, select, withColumn) ne déclenchent **aucun calcul immédiat**. Spark se contente de les enregistrer dans un plan logique sous forme de graphe (*DAG*).
- Seules les **actions** (comme count(), show(), write()) déclenchent l'exécution réelle du calcul.

Vérifions ce comportement en enchaînant des transformations sans action :

```
from pyspark.sql import functions as F

big = spark.range(0, 2_000_000, 1, 8).withColumn("bucket", F.col("id") % 5)

# Transformations uniquement
step1 = big.filter(F.col("id") % 2 == 0)
step2 = step1.select("id", "bucket")
step3 = step2.groupBy("bucket").count()

print("Transformations déclarées, mais aucun calcul n'a encore eu lieu !")
```

À présent, déclenchez l'exécution en appelant l'action `show()` :

```
step3.show()
```

```
big = spark.range(0, 2_000_000, 1, 8).withColumn("bucket", F.col("id") % 5)

# Transformations uniquement
step1 = big.filter(F.col("id") % 2 == 0)
step2 = step1.select("id", "bucket")
step3 = step2.groupBy("bucket").count()

print("Transformations déclarées, mais aucun calcul n'a encore eu lieu !")
```

```
Transformations déclarées, mais aucun calcul n'a encore eu lieu !
```

```
[9]: step3.show()
```

```
+------+------+
|bucket| count|
+------+------+
|     2|200000|
|     4|200000|
|     0|200000|
|     1|200000|
|     3|200000|
+------+------+
```

Dès l'appel de l'action, l'optimiseur interne (*Catalyst*) analyse l'ensemble du DAG, optimise le plan d'exécution physique et lance le calcul à travers les partitions.

8. Observer l'exécution via la Spark UI

Tant que votre session Spark est active, ouvrez votre navigateur à l'adresse http://localhost:4040.

Explorez les onglets clés :

1. **Jobs :** Visualisez l'historique de chaque action déclenchée et sa durée d'exécution.
2. **Stages :** Observez comment les jobs sont découpés en étapes. Chaque **Shuffle** (échange de données à travers le réseau provoqué par un groupBy ou une jointure) engendre la création d'un nouveau *stage*.
3. **Executors :** Vérifiez l'allocation des ressources et l'utilisation de la mémoire sur vos cœurs de calcul.

